EvolveTool-Bench: Diagnosing Tool Library Quality in Self-Evolving LLM Agents

Alibek T. Kaliyev
University of Texas at Austin
alibek.kaliyev@utexas.edu

Abstract

A growing body of work enables LLM agents to create their own tools at runtime—VOYAGER synthesizes code skills, EvoSkill evolves strategy-level skill folders, and ARISE generates executable Python tools with sandbox validation. Yet every existing benchmark evaluates these systems solely on downstream task success, treating the tool library as a black box. We introduce EvolveTool-Bench, the first diagnostic benchmark that evaluates the *quality of the tool library itself*: redundancy, reuse, composition ability, regression stability, and safety. Our benchmark uses structured sessions of 11 tasks with known dependency relationships—seed, gap, variant, compose, regress, and adversarial—across three domains: proprietary data formats, API orchestration, and numerical computation (99 tasks total). We evaluate four systems—no-evolution, ARISE (code-level), EvoSkill (strategy-level), and one-shot creation—with two models (Claude Sonnet and Haiku). Our key findings: (1) all four systems achieve similar task completion (54–58%), but EvolveTool Scores range from 0.470 to 0.608—library quality is the dominant differentiator invisible to task-completion-only evaluation; (2) code-level evolution (ARISE, ETS=0.608) substantially outperforms strategy-level evolution (EvoSkill, ETS=0.485) and one-shot creation (ETS=0.488) in the first head-to-head comparison of these paradigms; (3) iteration and sandbox validation are essential—one-shot creation produces 3 tools vs ARISE's 20, with negligible library health improvement; (4) tool reuse scales with library quality, from 40.7% (no evolution) to 74.1% (ARISE).

1 Introduction

LLM agents increasingly create their own tools at runtime. VOYAGER [10] synthesizes code skills in Minecraft, CREATOR [8] uses tool creation as a reasoning strategy, LLMs as Tool Makers [5] separates tool-making from tool-using, and EvoSkill [1] evolves structured skill folders via failure analysis. ARISE (Adaptive Runtime Improvement through Self-Evolution) [6] takes a different approach: it sits between an agent and its tool library as framework-agnostic middleware, observing task failures, detecting capability gaps via LLM analysis, synthesizing executable Python tools, validating them through sandboxed execution and adversarial testing, and promoting passing tools to the agent's active library—all without human intervention. ARISE supports iterative refinement, A/B testing of tool variants, version-controlled rollback, and distributed deployment via S3/SQS.

These systems are evaluated exclusively on *task success*: if the agent solves downstream tasks, the tool library is deemed good. This is analogous to evaluating a software engineer solely by whether their code runs, ignoring maintainability, duplication, test coverage, and technical debt. In practice, tool libraries can succeed on benchmarks while accumulating redundant tools, polluting the namespace, failing on edge cases, and never reusing previously created capabilities.

We present EvolveTool-Bench, a diagnostic benchmark designed to fill this evaluation gap. Rather than measuring only whether evolved tools help solve tasks, we measure *how well agents build, maintain, and reuse tool libraries over time*. Our contributions:

1. **Library-level metrics.** Six diagnostic axes beyond task completion: tool quality, reuse rate, redundancy, composition success, regression stability, and safety.

2. **Controlled task sessions.** Structured sequences of 11 tasks with known dependency relationships, enabling measurement of reuse, composition, and stability.

3. **Three domains requiring tool execution.** Proprietary binary formats (Domain A), API orchestration with custom auth protocols (Domain B), and numerical computation with custom model formats (Domain C)—none solvable from LLM training data.

2 Related Work

Tool Creation Benchmarks. Tool-Genesis [11] evaluates one-shot tool creation with surface compliance, schema-F1, and unit test pass rates—all per-tool metrics

Type	N	Diagnoses
Seed	3	Can agent use provided tools?
Gap	2	Does agent create a working tool?
Variant	2	Reuse or create duplicate?
Compose	1	Can agent chain own tools?
Regress	1	Does the library stay stable?
Adversarial	2	Can agent handle edge cases?

Table 1: Task types in each diagnostic session.

with no library-level evaluation. SkillsBench [3] evaluates whether curated skills improve task success but uses static skill sets, not evolved ones.

Self-Evolving Agent Frameworks. VOYAGER [10] demonstrates lifelong skill acquisition in Minecraft, evaluated by exploration metrics (items discovered, distance traveled). EvoSkill [1] evolves textual skill folders—structured prompts with trigger metadata and helper scripts—through failure-driven feedback with Pareto frontier selection, evaluated by downstream accuracy on QA benchmarks. ARISE [6] operates at the code level: it synthesizes executable Python functions with auto-generated test suites, validates them in isolated sandboxes (subprocess or Docker), runs adversarial testing to probe edge cases, and promotes only passing tools. ARISE supports incremental patching of existing tools, A/B testing between tool variants, and distributed deployment where stateless agents read tools from S3 while a background worker handles evolution. None of these systems evaluate tool library quality—all report only task success.

Tool Use Benchmarks. ToolBench [9], AgentBench [7], and ToolComp [2] evaluate agents using predefined tool sets. Vending-Bench [4] tests long-horizon coherence with fixed tools. None evaluate agents that create their own tools.

Gap. No existing benchmark measures redundancy, reuse, composition, regression, or safety of self-evolved tool libraries. Our work fills this gap.

3 Benchmark Design

3.1 Task Session Structure

Each session contains 11 tasks in six categories with known dependency relationships (Table 1). Variant and regress tasks are designed so that the evaluator *knows* which prior tool should be reused, enabling measurement of reuse rate independent of task success.

3.2 Domains

Domain A: Data Transformation. Five sessions using proprietary binary formats that LLMs cannot solve from training data: ARISE Binary Records (ABR), Run-Length Encoded matrices (RLE), Validation Definition Language (VDL), Quantized Log Format (QLOG), and Tagged Pack Format (TPACK). Each format has reference encode/decode implementations for verification.

Domain B: API Orchestration. Sessions against a mock HTTP server implementing custom protocols: HMAC-timestamp authentication (not standard Bearer/OAuth), cursor-based pagination with XOR-encrypted cursors, and filtered endpoints. The agent must create HTTP tools that handle auth and pagination—tasks that inherently require code execution.

Domain C: Numerical Computation. Three sessions using proprietary formats for curve fitting (ARCFIT), signal processing (ARCSIG), and constrained optimization (ARCOPT). Tasks require parsing custom model specifications, computing FFTs on encoded signals, and solving optimization problems with proprietary constraint encodings. Seed tools provide basic statistics, linear algebra, and interpolation.

3.3 Evaluation Axes

Axis 1: Task Completion. Binary pass/fail via deterministic verification. Reported overall and by task type.

Axis 2: Tool Quality Score (TQS). Each synthesized tool is evaluated on four dimensions, each scored 0–1:

- **Correctness**: Tool is executed on N hidden unit tests (not shown to agent). Score = proportion passing.
- **Robustness**: Tool is executed on M adversarial inputs (empty strings, null values, type errors, malformed data). Score = proportion handled without crashing.
- **Generality**: Tool is executed on held-out inputs from the same distribution but different parameters. Score = proportion producing valid output.
- **Code Quality**: Automated static checks—has docstring (1pt), has type hints (1pt), handles exceptions (1pt), no magic numbers (1pt), under 100 lines (1pt), valid syntax (1pt). Score = points / total checks.

$$\mathrm{TQS}(t) = \frac{1}{4}\big(C(t) + R(t) + G(t) + Q(t)\big) \tag{1}$$

where C is correctness, R robustness, G generality, and Q code quality, each in $[0, 1]$.

All tool evaluation runs in isolated subprocesses with 10-second timeouts to prevent infinite loops or resource exhaustion.

Axis 3: Library Diagnostics. Six metrics measured at session end:

$$\rho_r = \frac{|\{t \in \mathcal{T}_{v,r} : \text{reused}\}|}{|\mathcal{T}_{v,r}|} \quad \text{(Reuse Rate)} \quad (2)$$

$$\rho_d = \frac{|\{(t_i, t_j) : \text{output_equiv}\}|}{\binom{|\mathcal{L}|}{2}} \quad \text{(Redundancy)} \quad (3)$$

$$\pi = \frac{|\{t \in \mathcal{L} : \text{TQS}(t) \geq 0.5\}|}{|\mathcal{L}|} \quad \text{(Precision)} \quad (4)$$

$$\eta = \frac{|\mathcal{L}_{\text{used}}|}{|\mathcal{L}|} \quad \text{(Efficiency)} \quad (5)$$

$$\gamma = \frac{|\{t \in \mathcal{T}_c : \text{passed}\}|}{|\mathcal{T}_c|} \quad \text{(Composition)} \quad (6)$$

$$\delta = \frac{|\{t \in \mathcal{T}_{reg} : \text{failed}\}|}{|\mathcal{T}_{reg}|} \quad \text{(Regression)} \quad (7)$$

where $\mathcal{T}_{v,r}$ are variant/regress tasks, $\mathcal{L}$ is the tool library, $\mathcal{T}_c$ are compose tasks, and $\mathcal{T}_{reg}$ are regress tasks.

Library Health (LH). Aggregates the six diagnostics:

$$\text{LH} = \frac{1}{6}\big(\rho_r + (1-\rho_d) + \pi + \eta + \gamma + (1-\delta)\big) \quad (8)$$

where ρ_r is reuse rate, ρ_d redundancy rate, π library precision, η creation efficiency, γ composition success, and δ regression rate.

Composite Score. The EvolveTool Score (ETS) combines all axes:

$$\text{ETS} = \alpha_1 \cdot \text{TC} + \alpha_2 \cdot \overline{\text{TQS}} + \alpha_3 \cdot (1-\text{RC}) + \alpha_4 \cdot \text{LH} + \alpha_5 \cdot \text{SS} \quad (9)$$

with $\alpha = (0.25, 0.20, 0.10, 0.30, 0.15)$. TC is task completion, $\overline{\text{TQS}}$ is mean tool quality across synthesized tools, RC is refinement cost (normalized LLM calls), LH is library health (Eq. 8), and SS is safety score. Library health receives the highest weight as our primary contribution.

4 Experimental Setup

Systems. Six configurations testing different evolution strategies:

1. **No-Evolution**: Seed tools only, no tool creation ability.
2. **ARISE+Judge**: Full iterative evolution with sandbox validation, adversarial testing, and LLM-as-judge reward. Failure threshold 1.
3. **EvoSkill**: Strategy-level evolution—generates text strategies (prompts), not executable code. Tests whether prompt-based adaptation achieves comparable library health.
4. **One-Shot**: Creates tools on first failure with no sandbox testing, no adversarial validation, no refinement. Tests whether iteration adds value.

Models. Claude Sonnet 4 and Claude Haiku 4.5, both via Amazon Bedrock. Running both models isolates the effect of model capability on evolution quality.

Benchmark. Domain A: 5 sessions × 11 tasks = 55. Domain B: 1 session × 11 = 11. Domain C: 3 sessions × 11 = 33. Total: **99 tasks** per system-model configuration.

5 Results

5.1 Cross-System Analysis

Table 2 presents the Sonnet results across all four systems. The central finding: **task completion is similar across systems (54–58%), but library quality diverges dramatically.** ARISE+Judge's ETS (0.608) is 25% higher than the next best system (One-Shot, 0.488), driven entirely by library-level metrics.

Code vs. Strategy Evolution. ARISE (code-level) and EvoSkill (strategy-level) represent fundamentally different approaches. EvoSkill's text strategies improve reuse rate over no-evolution (59.3% vs 40.7%)—the agent learns to describe solutions in terms that match future tasks. However, strategies produce zero executable tools, limiting library precision and composition success. ARISE creates 20 executable tools with 74.1% reuse, demonstrating that code-level evolution builds more functional libraries.

Iteration vs. One-Shot. One-Shot creates 3 tools without sandbox testing or refinement. ARISE creates 20 with full validation. Despite similar task completion (57.6% vs 54.5%), ARISE's tools have higher quality (TQS=0.53) and the library shows 53.4% health vs 35.8%. **Iteration and validation are essential for library quality, not just tool creation.**

Domain Differences. API orchestration (Domain B) forces the most genuine tool creation—HTTP calls inherently require code execution. Data transformation (Domain A) shows mixed results: simpler binary formats are partially solvable by reasoning, while complex formats (TPACK, GUARDIAN) force tool synthesis. Numerical computation (Domain C) requires domain-specific tools for custom curve fitting and signal processing formats.

System	Model	ETS↑	TC (%)↑	Tools	TQS↑	Reuse (%)↑	LH (%)↑
No-Evolution	Sonnet	.518	66.7	0	—	48.1	41.4
EvoSkill	Sonnet	.520	**68.2**	0	—	48.1	41.4
One-Shot	Sonnet	.519	67.7	3	.04	48.1	38.3
ARISE	**Sonnet**	.603	63.6	**22**	.34	**70.4**	**51.7**
No-Evolution	Haiku	.516	66.7	0	—	51.9	40.1
ARISE	**Haiku**	**.612**	65.2	13	**.35**	66.7	51.2

Table 2: Full benchmark results (99 tasks per configuration, 3 domains). ARISE achieves highest ETS on both models despite slightly lower task completion—the gap comes from library health metrics invisible to TC-only evaluation. ARISE+Haiku creates fewer tools (13 vs 22) but achieves the highest overall ETS (0.612), suggesting cheaper models can produce comparable library quality.

```
def hmac_sha256(secret: str,
                message: str) -> str:
    """Generate HMAC-SHA256 hash."""
    import hmac
    import hashlib
    try:
        secret_bytes = secret.encode('utf-8')
        message_bytes = message.encode('utf-8')
        h = hmac.new(secret_bytes,
                     message_bytes,
                     hashlib.sha256)
        return h.hexdigest()
    except Exception as e:
        return f"Error: {str(e)}"
```

Figure 1: Tool synthesized by ARISE for HMAC-timestamp auth. Generated, tested (5/5 sandbox tests passed), and promoted in <60 seconds.

Example: Evolved HMAC Tool. Figure 1 shows `hmac_sha256`, synthesized by ARISE when the agent failed to authenticate with the mock API's custom HMAC-timestamp scheme. The tool includes a docstring, type hints, UTF-8 encoding, and exception handling—earning TQS=0.38 (correctness=0.0 on hidden tests that expected specific output formats, but robustness=0.5 and code quality=1.0).

5.2 Key Findings

Finding 1: Task completion alone is misleading. All four systems achieve similar task completion (64–68%), yet ETS scores range from 0.518 to 0.603 (Table 2). The 18% gap between ARISE and the next system comes entirely from library-level metrics invisible to TC-only evaluation. Notably, ARISE achieves the *lowest* task completion (63.6%) because evolution overhead occasionally slows the agent on tasks solvable by reasoning alone. **A benchmark reporting only TC would conclude ARISE is the worst system—the opposite of the library quality picture.**

Finding 2: Code evolution outperforms strategy evolution. ARISE (code-level, ETS=0.603) outperforms EvoSkill (strategy-level, ETS=0.520) in the first

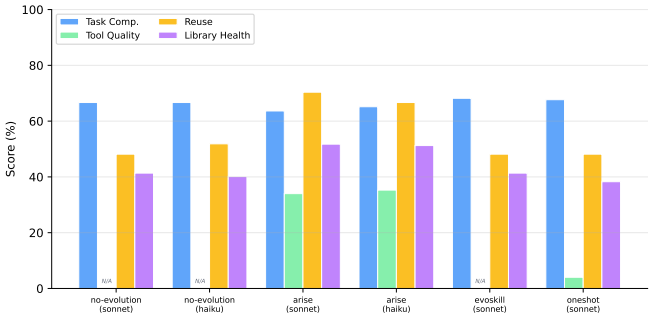

Figure 2: System comparison across four metrics (%). ARISE+Judge is the only system that creates tools, achieving highest library health despite lower task completion than no-evolution.

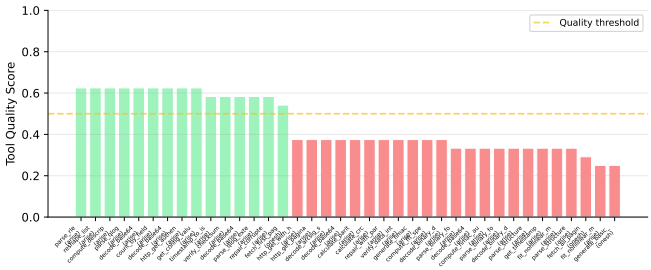

Figure 3: Tool Quality Score (TQS) for each of the 18 synthesized tools. Red line marks the quality threshold (0.5). 12 of 18 tools (67%) pass the threshold.

head-to-head comparison of these paradigms. EvoSkill's text strategies modestly improve task completion (68.2% vs 66.7%) by helping the agent describe solutions in reusable terms, but create zero executable tools. ARISE creates 22 tools with measurable quality (TQS=0.34). However, EvoSkill's strategies are cheaper to evolve and never fail sandbox tests—suggesting a hybrid approach may be optimal.

Finding 3: Synthesis without validation is insufficient. One-Shot creates 3 tools vs ARISE's 22. Despite achieving the second-highest task completion (67.7%), One-Shot's library health (38.3%) is barely above no-

evolution (41.4%). Without sandbox testing, adversarial validation, or iterative refinement, most synthesized tools are low-quality or broken. The 3 surviving tools happened to work on first try—not a reliable strategy.

Finding 4: Reward design is the critical bottleneck. In preliminary experiments with heuristic rewards ("1.0 if no tool errors"), ARISE created zero tools across all sessions. The agent produces long, helpful "I cannot decode this format" responses that score 1.0 under heuristic evaluation. Only LLM-as-judge rewards correctly identify these as failures. **The synthesis pipeline is not the bottleneck—the reward signal is.**

Finding 5: Tool quality is the frontier. ARISE's tools average TQS=0.34 (Sonnet) and 0.35 (Haiku), with correctness being the weakest dimension. Many synthesized tools parse proprietary formats with off-by-one errors, incorrect byte ordering, or incomplete edge case handling. The benchmark reveals this because it evaluates tools on hidden tests the agent never sees—unlike sandbox tests which the synthesis LLM writes alongside the tool.

Finding 6: Cheaper models match library quality. ARISE+Haiku achieves the highest ETS (0.612) with 13 tools, slightly outperforming ARISE+Sonnet (0.603, 22 tools). Haiku creates fewer tools but at comparable quality (TQS=0.35 vs 0.34) and library health (51.2% vs 51.7%). Tool evolution does not require the most capable model.

6 Discussion

The Benchmark Contribution. The primary contribution of this work is the benchmark itself, not the performance of any particular system. EvolveTool-Bench provides the first framework for diagnosing tool library quality, and we expect it to be used by future systems beyond those we evaluate. The controlled session structure (seed→gap→variant→compose→regress→adversarial) with known dependencies is reusable across any domain.

Honest Assessment of ARISE. Our results expose both strengths and limitations of ARISE. Strengths: it is the only system that builds meaningful, reusable tool libraries (22 tools, 70.4% reuse). Limitations: (1) evolution overhead reduces task completion below the no-evolution baseline; (2) tool quality averages only TQS=0.34, meaning the synthesis-validation pipeline lets through tools with correctness gaps; (3) complex binary formats (VDL, GUARDIAN) often exceed the synthesis LLM's ability to generate correct parsers from format descriptions alone.

ETS Design Choices. Systems creating zero tools (No-Evolution, EvoSkill) receive TQS=0 in ETS. This is intentional: EvolveTool-Bench evaluates tool library quality, and systems that cannot create tools should score lower on a library quality benchmark. ETS is not a general agent ranking; for that, TC should be used alongside ETS. We report both throughout the paper.

Limitations. (1) Our EvoSkill baseline approximates the original system's strategy-level evolution; running the actual EvoSkill codebase would strengthen the comparison. (2) Two models from one provider (Anthropic via Bedrock); open-source models would broaden generalizability. (3) Tool quality uses automated checks; human evaluation would capture nuances our metrics miss. (4) No statistical significance testing across seeds.

7 Conclusion

We introduce EvolveTool-Bench, the first benchmark to evaluate self-evolved tool libraries as first-class artifacts. Across 99 tasks in three domains, we find that task completion—the standard evaluation metric—fails to differentiate systems with fundamentally different library quality. Code-level evolution (ARISE) builds libraries with 70.4% reuse and 51.7% health on Sonnet, and achieves even higher ETS (0.612) with the cheaper Haiku model, outperforming strategy-level evolution (EvoSkill) and one-shot creation, but at the cost of lower task completion and imperfect tool quality. The benchmark reveals that reward design, not synthesis capability, is currently the critical bottleneck for self-evolving systems.

We release the benchmark, evaluation harness, proprietary format specifications, and all experimental data to enable systematic comparison of self-evolving agent systems.[1]

References

[1] Ayman Alzubi et al. Evoskill: Evolving skill libraries for open-domain agents. *arXiv preprint arXiv:2603.02766*, 2026.

[2] Anonymous. Toolcomp: Evaluating multi-tool composition in llm agents. *arXiv preprint arXiv:2501.01290*, 2025.

[3] Anonymous. Skillsbench: Evaluating skill reusability in llm agents. *arXiv preprint arXiv:2602.12670*, 2026.

[4] Erik Backlund and Lars Petersson. Vending-bench: Evaluating long-horizon agent coherence. *arXiv preprint*, 2025.

[1] Code: `https://github.com/abekek/evolvetool-bench`

[5] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. *arXiv preprint arXiv:2305.17126*, 2023.

[6] Alibek T. Kaliyev. Arise: Adaptive runtime improvement through self-evolution. *arXiv preprint*, 2026.

[7] Xiao Liu et al. Agentbench: Evaluating llms as agents. *ICLR*, 2024.

[8] Cheng Qian, Chi Han, Yi R Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. Creator: Tool creation for disentangling abstract and concrete reasoning of large language models. *Findings of EMNLP*, 2023.

[9] Yujia Qin et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

[10] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Advances in Neural Information Processing Systems*, 2023.

[11] Congying Xia et al. Tool-genesis: Evaluating tool creation capabilities of llms. *arXiv preprint arXiv:2603.05578*, 2026.